

MONGODB TOÀN TẬP CHO KỸ SƯ SQL (POSTGRESQL / MYSQL)

GIẢI MÃ KIẾN TRÚC DỮ LIỆU DỰ ÁN VELSTRONG BOOKSTORE (HAUI MASTER)

MỤC LỤC

- 1. [Chuyển Đổi Tư Duy: Từ Relational \(PostgreSQL/MySQL\) Sang Document Store \(MongoDB\)](#) - 1.1. Bảng Quy Đổi Thuật Ngữ Đối Chiếu 1-1 - 1.2. Triết Lý Thiết Kế Schema: Write-Optimized (SQL) vs Read-Optimized (MongoDB) - 1.3. Bài Toán Quyết Định: Khi Nào Embed (Nhúng) vs Khi Nào Reference (Tham Chiếu)? - 1.4. Động Cơ Lưu Trữ WiredTiger: B-Tree, Cache, Checkpoint & Journal - 1.5. Giao Dịch Đa Tài Liệu (Multi-Document ACID Transactions)
- 2. [Kiến Trúc MongoDB Đặc Thù Trong Dự Án Velstrong Bookstore](#) - 2.1. Động Lực Lựa Chọn MongoDB Cho Chương Trình Thạc Sĩ Dữ Liệu Nâng Cao - 2.2. Tính Thuần Khiết Kiến Trúc Lục Giác (Zero Mongo Dependency In Domain Core) - 2.3. Cơ Chế Khóa Chính Tự Tăng (Sequence Pattern Qua `_mongo_sequences`) - 2.4. Kiểm Soát Đồng Thời: Khóa Lạc Quan (`saveVersioned`) vs Khóa Bi Quan (`_mongoLock`) - 2.5. Spring Data MongoDB & Quản Lý Giao Dịch Phân Tán Với `MongoTransactionManager`
- 3. [Giải Mã Chi Tiết Toàn Bộ 34 Collections & Documents Trong Dự Án](#) - 3.1. Nhóm 1: Hệ Thống & Hạ Tầng (2 Collections) - 3.2. Nhóm 2: Người Dùng, Phân Quyền & Địa Chỉ (7 Collections) - 3.3. Nhóm 3: Danh Mục Sách & Kho Bàn Sao Vật Lý (4 Collections) - 3.4. Nhóm 4: Giỏ Hàng & Khuyến Mãi (4 Collections) - 3.5. Nhóm 5: Đơn Hàng & Lịch Sử Trạng Thái (3 Collections) - 3.6. Nhóm 6: Thanh Toán & Đối Soát Ngân Hàng (3 Collections) - 3.7. Nhóm 7: Thuê Sách & Gói Hội Viên Đọc Sách (4 Collections) - 3.8. Nhóm 8: Tương Tác, Hỗ Trợ, Đánh Giá & Truyền Thông (7 Collections)
- 4. [So Sánh Các Mẫu Truy Vấn Điển Hình: SQL Query vs MongoDB Aggregation](#)
- 5. [Cẩm Nang Thực Chiến & Quy Tắc Thiết Kế Cho Kỹ Sư SQL](#)

1. CHUYỂN ĐỔI TƯ DUY: TỪ RELATIONAL (POSTGRESQL/MYSQL) SANG DOCUMENT STORE (MONGODB)

Là một kỹ sư đã dày dạn kinh nghiệm với PostgreSQL hoặc MySQL, bạn đã quen thuộc với mô hình bảng quan hệ (Relational Model), chuẩn hóa dữ liệu (Normal Forms từ 1NF đến 3NF), khóa ngoại (`FOREIGN KEY`) và câu lệnh `JOIN` . Khi bước chân vào thế giới **MongoDB (Document-Oriented Database)**, điều quan trọng nhất không phải là học cú pháp mới, mà là **thay đổi mô hình tư duy dữ liệu (Mental Model Shift)**.

1.1. Bảng Quy Đổi Thuật Ngữ Đối Chiếu 1-1

Khái Niệm Trong SQL (Postgres / MySQL)	Khái Niệm Tương Đương Trong MongoDB	Bản Chất & Điểm Khác Biệt Cốt Lõi
Database (Cơ sở dữ liệu)	Database (Cơ sở dữ liệu)	Tương đồng hoàn toàn. Chứa tập hợp các đối tượng dữ liệu.
Table (Bảng dữ liệu)	Collection (Tập hợp tài liệu)	Bảng SQL có cấu trúc tĩnh (Rigid Schema), mọi dòng phải tuân thủ đúng số cột đã định nghĩa DDL. Collection trong MongoDB có cấu trúc động (Dynamic/Polymorphic Schema), các document trong cùng một collection có thể có các trường dữ liệu khác nhau.

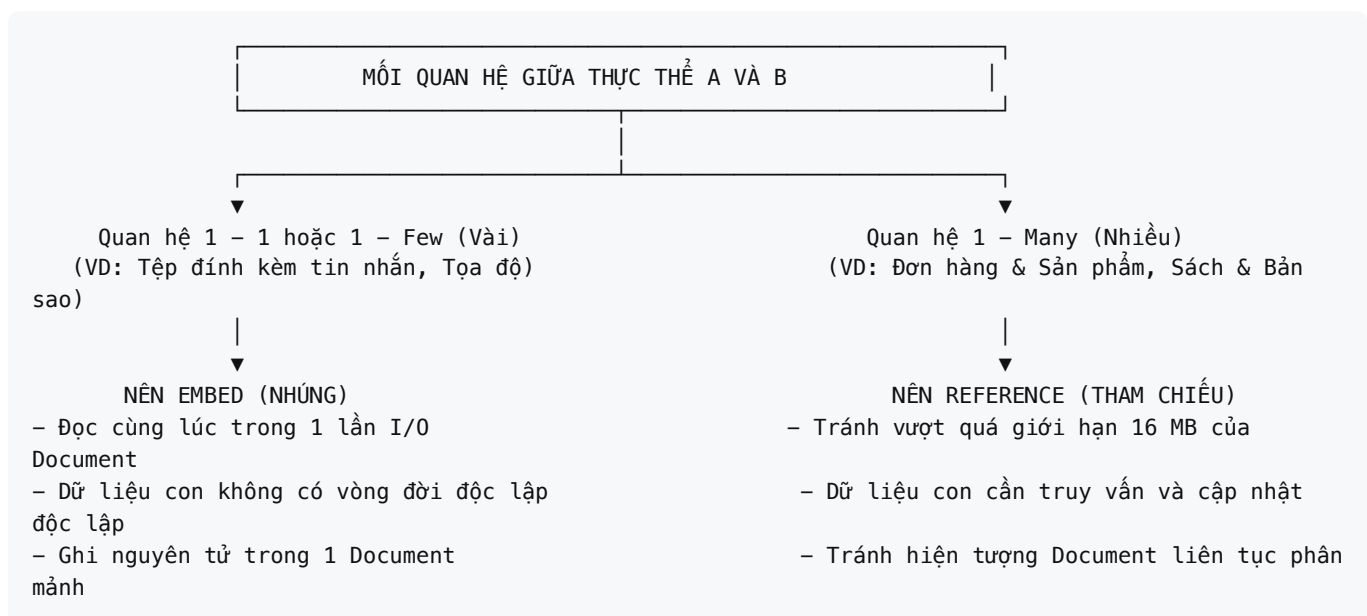
Khái Niệm Trong SQL (Postgres / MySQL)	Khái Niệm Tương Đương Trong MongoDB	Bản Chất & Điểm Khác Biệt Cốt Lõi
Row / Tuple / Record (Dòng dữ liệu)	Document (Tài liệu)	Dòng SQL là danh sách giá trị phẳng (flat scalar values). Document trong MongoDB là đối tượng BSON (Binary JSON) có cấu trúc cây, hỗ trợ dữ liệu lồng nhau nhiều tầng (Nested Objects) và danh sách mảng (Arrays). Giới hạn kích thước tối đa 1 document là 16 MB .
Column (Cột)	Field (Trường dữ liệu)	Cột SQL yêu cầu kiểu dữ liệu cố định từ trước. Trường trong MongoDB lưu trữ theo cặp Key-Value, có thể xuất hiện hoặc không xuất hiện giữa các document.
Primary Key (SERIAL , AUTO_INCREMENT)	_id Field (ObjectId hoặc Custom)	Mọi Document trong MongoDB bắt buộc có trường <code>_id</code> làm khóa chính duy nhất. Mặc định là <code>ObjectId</code> (12 bytes chứa timestamp, machine id, process id, counter). Có thể thay thế bằng kiểu số nguyên (<code>Long</code>) hoặc chuỗi tùy biến.
Foreign Key (REFERENCES)	Document Reference / DBRef	SQL có Database Engine ép buộc toàn vẹn tham chiếu (Referential Integrity constraints, <code>ON DELETE CASCADE</code>). Trong MongoDB, không có khóa ngoại ở tầng Database Engine ; tính toàn vẹn tham chiếu do tầng Ứng dụng (Application Layer) đảm bảo.
JOIN (INNER JOIN , LEFT JOIN)	Embedding (Nhúng) hoặc \$lookup	Trong SQL, liên kết dữ liệu bắt buộc thực hiện tại thời điểm truy vấn thông qua phép tính toán JOIN đắt đỏ. Trong MongoDB, triết lý ưu tiên là Nhúng sẵn dữ liệu liên quan vào cùng một Document ; trường hợp bắt buộc phân tách mới sử dụng toán tử <code>\$lookup</code> trong Aggregation Pipeline hoặc Application-level Join.
CREATE TABLE , ALTER TABLE	Dynamic Schema / JSON Schema Validator	MongoDB không bắt buộc DDL để tạo bảng. Collection tự động được tạo khi có thao tác chèn document đầu tiên. Tuy nhiên, MongoDB hỗ trợ JSON Schema Validation ở mức collection để ép buộc quy tắc nếu cần.
Index (B-Tree Index)	WiredTiger B-Tree Index	Tương đồng về cấu trúc toán học B-Tree. Tuy nhiên MongoDB hỗ trợ thêm: Multikey Index (đánh chỉ mục trực tiếp vào từng phần tử của mảng), Compound Index , Text Index , Geospatial Index (2dsphere) , TTL Index (tự động xóa tài liệu sau thời gian hết hạn).
GROUP BY , Aggregate Functions	Aggregation Framework Pipeline	Thay vì cú pháp khai báo SQL, MongoDB sử dụng đường ống biến đổi dữ liệu tuần tự đa tầng: <code>\$match</code> $\rightarrow$ <code>\$unwind</code> $\rightarrow$ <code>\$group</code> $\rightarrow$ <code>\$project</code> $\rightarrow$ <code>\$sort</code> $\rightarrow$ <code>\$limit</code> .
ACID Transactions	Multi-Document ACID Transactions	Trước phiên bản 4.0, MongoDB chỉ đảm bảo ACID trên 1 Document đơn lẻ. Từ phiên bản 4.0 trở đi , trên cụm Replica Set, MongoDB hỗ trợ hoàn chỉnh Multi-Document ACID Transactions tương tự Postgres/MySQL.

1.2. Triết Lý Thiết Kế Schema: Write-Optimized vs Read-Optimized

- **Thế giới SQL (Write-Optimized / Zero Redundancy):**
- Chuẩn hóa dữ liệu (Normalization) ra đời vào thập niên 1970 khi dung lượng ổ cứng còn cực kỳ đắt đỏ. Mục tiêu tối thượng của 3NF là **loại bỏ hoàn toàn dư thừa dữ liệu (No Redundancy)** và tránh dị thường khi cập nhật (Update Anomaly).
- Hệ quả: Khi đọc dữ liệu phức tạp (ví dụ: màn hình chi tiết đơn hàng), hệ thống phải thực hiện 5-10 phép JOIN giữa các bảng orders, order_items, users, addresses, books, payments. Chi phí CPU và bộ nhớ của RDBMS tăng vọt khi quy mô hệ thống mở rộng.
- **Thế giới MongoDB (Read-Optimized / Access-Pattern Driven):**
- Dung lượng lưu trữ ngày nay rất rẻ, trong khi độ trễ truy vấn (Latency) và năng lực mở rộng ngang (Horizontal Scalability) là yếu tố sống còn.
- **Quy tắc vàng của NoSQL:** "Data that is accessed together should be stored together" (Dữ liệu thường xuyên được đọc cùng nhau thì nên được lưu trữ cùng nhau trong một Document).
- Thiết kế Schema trong MongoDB không bắt đầu từ thực thể tĩnh, mà bắt đầu từ **Mô thức truy vấn của ứng dụng (Application Access Patterns)**: Ứng dụng cần đọc màn hình này bằng một truy vấn duy nhất như thế nào?

1.3. Bài Toán Quyết Định: Khi Nào Embed (Nhúng) vs Khi Nào Reference (Tham Chiếu)?

Đây là câu hỏi cốt lõi của mọi kỹ sư SQL khi chuyển sang MongoDB. Hãy áp dụng bộ quy tắc 5 mức quan hệ sau:



1. **Quan hệ 1 - 1:** Hầu như luôn **Embed**.
2. **Quan hệ 1 - Few (1 đến vài chục):** **Embed** trực tiếp dưới dạng Sub-document hoặc Array of Sub-documents (ví dụ: các ảnh đính kèm của 1 tin nhắn hỗ trợ, danh sách các thẻ tag của một cuốn sách).
3. **Quan hệ 1 - Many (1 đến hàng ngàn):** **Reference bằng ID** (ví dụ: Một người dùng có hàng ngàn đơn hàng $\rightarrow$ Lưu `userId` trong document `orders`).
4. **Quan hệ 1 - Squillions (1 đến vô hạn):** Bắt buộc **Reference ngược** từ phía con trở về phía cha để tránh mảng con bị phình to vô hạn (Unbounded Array Anti-pattern).
5. **Quan hệ Many - Many (N - N):** Tùy thuộc kích thước:
 - Nếu tập nhỏ (dưới 50): Lưu mảng ID tham chiếu (ví dụ: `book.categoryIds = [1, 5, 8]`).
 - Nếu tập lớn và cần lưu kèm siêu dữ liệu liên kết: Sử dụng **Link Collection** (Tương đương bảng nối trong SQL, ví dụ: `book_categories`, `user_roles`, `role_permissions` trong dự án này).

1.4. Động Cơ Lưu Trữ WiredTiger: B-Tree, Cache, Checkpoint & Journal

MongoDB sử dụng động cơ lưu trữ mặc định **WiredTiger**: - **Bộ nhớ đệm (WiredTiger Internal Cache)**: - Chiếm khoảng $50\% \text{ RAM} - 1\text{ GB}$ của máy chủ. Toàn bộ dữ liệu đọc và ghi đều diễn ra trên RAM cache trước khi ghi xuống đĩa cứng. - **Checkpointing**: - Cứ mỗi **60 giây** (hoặc khi dung lượng log đạt 2GB), WiredTiger tạo một điểm kiểm tra (Checkpoint). Toàn bộ dữ liệu thay đổi trên RAM được xả (flush) xuống đĩa cứng thành tệp dữ liệu nhất quán. - **Write-Ahead Logging (Journaling)**: - Tương tự như Write-Ahead Log (WAL) của PostgreSQL hay Redo Log của MySQL InnoDB. - Mọi thao tác ghi trước khi cập nhật vào Cache đều được ghi tuần tự vào Journal trên đĩa cứng. Nếu máy chủ bị cúp điện đột ngột giữa 2 chu kỳ Checkpoint, WiredTiger dùng Journal để phục hồi nguyên vẹn mọi giao dịch đã cam kết. - **Mức độ Khóa (Concurrency & Lock Granularity)**: - WiredTiger hỗ trợ **Khóa cấp độ Tài liệu (Document-level Concurrency)**. Hai luồng cập nhật vào hai document khác nhau trong cùng một collection sẽ chạy song song hoàn toàn mà không hề chặn nhau.

1.5. Giao Dịch Đa Tài Liệu (Multi-Document ACID Transactions)

Kể từ MongoDB 4.0, MongoDB hỗ trợ giao dịch đa tài liệu đầy đủ 4 tính chất ACID trên cụm **Replica Set**: - **Atomicity (Nguyên tử)**: Toàn bộ các thao tác ghi trên nhiều collection khác nhau hoặc cùng thành công, hoặc toàn bộ được hủy bỏ (Rollback) không để lại dấu vết. - **Consistency (Nhất quán)**: Dữ liệu luôn tuân thủ các chỉ mục duy nhất và JSON Schema Validation. - **Isolation (Cô lập)**: Hỗ trợ mức cô lập **Snapshot Isolation**. Các truy vấn bên ngoài giao dịch không bao giờ nhìn thấy dữ liệu đang thay đổi dở dang cho tới khi giao dịch được Commit. - **Durability (Bền vững)**: Sử dụng cơ chế ghi đồng thuận $w: \text{majority}$ trên đa số các node của Replica Set để đảm bảo dữ liệu không bị mất ngay cả khi node chính (Primary) gặp sự cố.

2. KIẾN TRÚC MONGODB ĐẶC THÙ TRONG DỰ ÁN VELSTRONG BOOKSTORE

2.1. Động Lực Lựa Chọn MongoDB Cho Chương Trình Thạc Sĩ Dữ Liệu Nâng Cao

Dự án Velstrong Bookstore được thiết kế cho học phần **Cơ sở dữ liệu nâng cao** (HaUI Master). Mục tiêu trọng tâm là: 1. Vận dụng kiến trúc CSDL phi quan hệ (NoSQL Document Database) trong bài toán thương mại điện tử thực tế có nghiệp vụ phức tạp (Mua bán kết hợp Thuê sách, Đối soát ngân hàng, Hỗ trợ trực tuyến). 2. Triển khai mô hình **Cụm bản sao (Replica Set rs0)** để kích hoạt tính năng **Giao dịch phân tán đa tài liệu (Multi-document ACID Transactions)** trên MongoDB. 3. Giải quyết bài toán bảo toàn toàn vẹn dữ liệu trong môi trường Document Store: Tự sinh khóa tuần tự, khóa lạc quan (Optimistic Locking) và khóa bi quan (Pessimistic Locking).

2.2. Tính Thuần Khiết Kiến Trúc Lục Giác (Zero Mongo Dependency In Domain Core)

Một trong những điểm độc đáo nhất của dự án này nằm ở cách tổ chức mã nguồn: - **Quy tắc bất biến của Hexagonal Architecture**: Thư mục `com.velstrong.bookstore.domain.*` phải **hoàn toàn tinh khiết**, không được chứa bất kỳ annotation nào của Spring Data, JPA, Jackson hay MongoDB (`@Document`, `@Id`, `@Field`, `@DBRef`). - **Cách dự án hiện thực hóa**: Không dùng Spring Data Mongo Repository interfaces kế thừa `MongoRepository`. - Thay vào đó, tạo lớp cơ sở **MongoPersistenceSupport** gói gọn `MongoTemplate`. - Các adapter (`Mongo*PersistenceAdapter`) điều khiển trực tiếp tên collection bằng chuỗi ký tự (`String collection`), ánh xạ linh hoạt giữa Domain Model, JPA Entity và Document của Mongo.

2.3. Cơ Chế Khóa Chính Tự Tăng (Sequence Pattern Qua `_mongo_sequences`)

Trong PostgreSQL bạn có `BIGSERIAL`, trong MySQL bạn có `AUTO_INCREMENT`. Mặc định, MongoDB sử dụng `_id` là chuỗi 24 ký tự hex của `ObjectId`. Tuy nhiên, dự án Velstrong Bookstore lại sử dụng **khóa chính số nguyên Long tự tăng (1, 2, 3...)** cho toàn bộ 34 collections.

Tại sao phải làm như vậy? 1. **Tính tương thích kép (Dual-profile Compatibility)**: Hệ thống duy trì song song hai profile persistence: `mongodb` (mặc định) và `postgres` (fallback). Sử dụng ID dạng Long cho phép hai profile hoán

đổi cho nhau mà không làm thay đổi Domain Model hay REST API contract. 2. **Thân thiện với người dùng và API:** Khách hàng tra cứu đơn hàng `/don-hang/15` thay vì `/don-hang/64f12ab345cd7890ef123456`.

Cơ chế kỹ thuật đằng sau (Sequence Pattern): Hệ thống sử dụng collection đặc biệt mang tên `_mongo_sequences`. Mỗi collection có một dòng đếm giá trị:

```
{
  "_id": "orders",
  "value": 142
}
```

Khi cần cấp phát ID mới cho một bản ghi, hàm `nextId(collection)` trong `MongoPersistenceSupport` thực thi:

```
Query query = Query.query(Criteria.where("_id").is(collection));
Update update = new Update().inc("value", 1);
Document sequence = mongo.findAndModify(query, update,
    FindAndModifyOptions.options().upsert(true).returnNew(true), Document.class,
    "_mongo_sequences");
return ((Number) sequence.get("value")).longValue();
```

Ưu điểm vượt trội: Lệnh `findAndModify` với toán tử `$inc` của MongoDB là thao tác **nguyên tử tuyệt đối (Atomic operation)** ở cấp độ nhân cơ sở dữ liệu. Ngay cả khi có 1000 tiến trình đồng thời tạo đơn hàng, mỗi tiến trình đều nhận được một số ID duy nhất, đơn điệu tăng dần, không bao giờ bị trùng lặp hoặc sinh khoảng trống.

2.4. Kiểm Soát Đồng Thời: Khóa Lạc Quan vs Khóa Bi Quan Trên MongoDB

A. Khóa Lạc Quan (Optimistic Concurrency Control - OCC)

Áp dụng cho các thực thể quan trọng như `Order`, `Payment`, `BookCopy` thông qua trường `version`:

```
private <T> T saveVersioned(String collection, T entity) {
    ...
    // 1. Kiểm tra version hiện tại dưới CSDL
    if (expectedVersion != currentVersion) {
        throw new OptimisticLockingFailureException("Stale Mongo version for " +
            entity.getClass().getSimpleName());
    }
    // 2. Thay thế tài liệu với điều kiện version phải khớp
    UpdateResult result = mongo.replace(
        Query.query(Criteria.where("_id").is(id).and("version").is(currentVersion)),
        entity, new ReplaceOptions(), collection);
    if (result.getMatchedCount() == 0) {
        throw new OptimisticLockingFailureException("Concurrent Mongo update");
    }
    ...
}
```

Khi hai nhân viên cùng mở màn hình chỉnh sửa một đơn hàng tại phiên bản `version = 2`. Người thứ nhất bấm Lưu $\rightarrow$ Hệ thống cập nhật thành công và đẩy lên `version = 3`. Người thứ hai bấm Lưu $\rightarrow$ Điều kiện `version: 2` không còn khớp với bản ghi trên đĩa $\rightarrow$ `result.getMatchedCount() == 0` $\rightarrow$ Hệ thống từ chối cập nhật và ném ra ngoại lệ `OptimisticLockingFailureException`, ngăn chặn việc ghi đè dữ liệu vô tình (Lost Update).

B. Khóa Bi Quan (Pessimistic Locking / `SELECT FOR UPDATE`)

Áp dụng trong bài toán phân bổ sách thuê: Khi trong kho chỉ còn 1 cuốn sách duy nhất, làm thế nào để đảm bảo 2 khách hàng bấm thanh toán cùng một giây không bị gán cùng một cuốn sách? Trong PostgreSQL bạn viết: `SELECT * FROM book_copies WHERE book_id = :id AND status = 'AVAILABLE' LIMIT 1 FOR UPDATE` . Trong MongoDB, `MongoBookCopyPersistenceAdapter` hiện thực hóa bằng cơ chế tài tình:

```
public Optional<BookCopy> findFirstAvailableByBookIdForUpdate(Long bookId) {
    Query query = Query.query(new Criteria().andOperator(
        Criteria.where("bookId").is(bookId),
        Criteria.where("status").is(BookCopyStatus.AVAILABLE.name())))
        .with(Sort.by(Sort.Direction.ASC, "_id")).limit(1);

    // Thao tác ghi cập nhật _mongoLock bên trong transaction tạo ra xung đột ghi (Write Conflict)
    // với các giao dịch cạnh tranh khác, đạt được hiệu quả tương đương SELECT FOR UPDATE trong
    SQL.
    return findAndModify(COLLECTION, BookCopyJpaEntity.class, query,
        new Update().set("_mongoLock", UUID.randomUUID().toString()).map(this::toDomain);
    }
}
```

Bằng cách thực hiện lệnh `findAndModify` thay đổi trường `_mongoLock` ngay trong giao dịch `ClientSession` , WiredTiger Engine lập tức chiếm quyền khóa ghi độc quyền trên Document đó. Bất kỳ giao dịch nào khác cố gắng đọc/sửa document này sẽ bị chặn lại hoặc nhận lỗi `TransientTransactionError` để thử lại sau.

2.5. Spring Data MongoDB & Quản Lý Giao Dịch Phân Tán Với `MongoTransactionManager`

Trong cấu hình `MongoPersistenceConfig.java` :

```
@Configuration
@Profile("mongodb & !postgres")
public class MongoPersistenceConfig {
    @Bean
    public PlatformTransactionManager transactionManager(MongoDatabaseFactory mongoDatabaseFactory)
    {
        return new MongoTransactionManager(mongoDatabaseFactory);
    }
}
```

Nhờ khai báo `MongoTransactionManager` , annotation chuẩn của Spring Framework là `@Transactional` hoạt động hoàn hảo trên MongoDB y hệt như trên RDBMS: - Khi một service method được gắn `@Transactional` (ví dụ: `CreateOrderService.create()`) : 1. Spring mở một `ClientSession` mới trên cụm MongoDB Replica Set `rs0` . 2. Bắt đầu phiên giao dịch: `session.startTransaction()` . 3. Mọi thao tác lưu qua `mongo.save()` , `mongo.insert()` , `mongo.remove()` đều được truyền ngầm `ClientSession` này. 4. Nếu method thực thi trọn vẹn mà không có exception $\rightarrow$ Spring gọi `session.commitTransaction()` . Toàn bộ dữ liệu tại 7 collections được đẩy chính thức vào kho lưu trữ. 5. Nếu có bất kỳ lỗi nào (ví dụ: voucher hết hạn hoặc sách hết hàng) $\rightarrow$ Spring gọi `session.abortTransaction()` . Toàn bộ các thao tác ghi trước đó trong phiên đều bị tiêu hủy sạch sẽ, đảm bảo dữ liệu toàn vẹn 100%.

3. GIẢI MÃ CHI TIẾT TOÀN BỘ 34 COLLECTIONS & DOCUMENTS TRONG DỰ ÁN

Dưới đây là bảng phân tích toàn diện 34 Collections trong cơ sở dữ liệu MongoDB của dự án Velstrong Bookstore:

3.1. Nhóm 1: Hệ Thống & Hạ Tầng (Infrastructure Collections)

1. Collection: `_mongo_sequences`

- **Mục đích nghiệp vụ:** Lưu trữ và quản lý giá trị đếm tự tăng (Auto-increment counter) cho từng collection nghiệp vụ, thay thế cho cơ chế Sequence / Serial của RDBMS.
- **Cấu trúc Document mẫu:**

```
{
  "_id": "orders",
  "value": 254
}
```

- **Khóa chính (`_id`):** Chuỗi ký tự (String) tương ứng với tên của collection cần quản lý ID.
- **Chỉ mục:** Khóa chính `_id` mặc định.

2. Collection: `user_session_versions`

- **Mục đích nghiệp vụ:** Quản lý số phiên bản đăng nhập của từng tài khoản người dùng, hỗ trợ tính năng đăng xuất tức thì khỏi mọi thiết bị mà không cần cơ chế Token Blacklist nặng nề.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 18,
  "version": 3
}
```

- **Khóa chính (`_id`):** Kiểu số nguyên Long , tương ứng với `userId` của người dùng.
- **Chỉ mục:** Khóa chính `_id` mặc định.

3.2. Nhóm 2: Người Dùng, Phân Quyền & Địa Chỉ (User & Security Collections)

3. Collection: `users`

- **Mục đích nghiệp vụ:** Lưu trữ tài khoản người dùng (Khách hàng, Nhân viên, Thủ kho, Quản trị viên).
- **Cấu trúc Document mẫu:**

```
{
  "_id": 1,
  "username": "customer_demo",
  "email": "customer@velstrongbook.asia",
  "passwordHash": "$2a$10$N.Z0Dq8V...",
  "fullName": "Nguyễn Văn Đọc",
  "phoneNumber": "0912345678",
  "iamId": "iam_usr_99812",
  "isActive": true,
  "emailVerified": true,
  "createdAt": "2026-08-01T08:00:00.000Z",
  "updatedAt": "2026-09-01T10:30:00.000Z"
}
```

- **Chỉ mục (Indexes):**
- `uk_users_username` (Unique): Bắt buộc username là duy nhất.
- `uk_users_email` (Unique): Bắt buộc email là duy nhất.
- `uk_users_iamId` (Unique, Sparse): Định danh đồng bộ hệ thống IAM ngoài (nếu có).

4. Collection: `user_addresses`

- **Mục đích nghiệp vụ:** Sổ địa chỉ giao hàng của khách hàng (Mỗi quan hệ 1 - N với `users`).
- **Cấu trúc Document mẫu:**

```
{
  "_id": 101,
  "userId": 1,
  "recipientName": "Nguyễn Văn Đọc",
  "phoneNumber": "0912345678",
  "streetAddress": "Số 298 đường Cầu Diễn",
  "ward": "Phường Minh Khai",
  "district": "Quận Bắc Từ Liêm",
  "city": "Thành phố Hà Nội",
  "isDefault": true,
  "createdAt": "2026-08-05T14:20:00.000Z"
}
```

- **Chỉ mục (Indexes):**
- `idx_user_addresses_userId_createdAt` : Tối ưu hóa truy vấn danh sách địa chỉ của người dùng sắp xếp theo thời gian tạo mới nhất.

5. Collection: `roles`

- **Mục đích nghiệp vụ:** Lưu trữ danh mục 4 vai trò hệ thống (`CUSTOMER` , `SALES_STAFF` , `WAREHOUSE_MANAGER` , `ADMIN`).
- **Cấu trúc Document mẫu:**

```
{
  "_id": 1,
  "code": "CUSTOMER",
  "name": "Customer",
  "description": "Customer account with shopping and rental rights"
}
```

- **Chỉ mục:** `uk_roles_code` (Unique).

6. Collection: `permissions`

- **Mục đích nghiệp vụ:** Lưu trữ 23 quyền hạn nguyên tử của hệ thống.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 1,
  "code": "book:read"
}
```


- **Chỉ mục:** `uk_permissions_code` (Unique).

7. Collection: `user_roles`

- **Mục đích nghiệp vụ:** Bảng nối (Link collection) thể hiện quan hệ N - N giữa Người dùng và Vai trò.
- **Cấu trúc Document mẫu:**

```
{
  "_id": "1:1",
  "userId": 1,
  "roleId": 1
}
```

- **Chỉ mục:** `uk_user_roles_userId_roleId` (Unique Compound).

8. Collection: `user_permissions`

- **Mục đích nghiệp vụ:** Cấp quyền đặc thù trực tiếp cho từng người dùng kèm thời gian hết hạn (`expiresAt`).
- **Cấu trúc Document mẫu:**

```
{
  "_id": "1:15",
  "userId": 1,
  "permissionId": 15,
  "expiresAt": "2026-12-31T23:59:59.000Z"
}
```

- **Chỉ mục:** `uk_user_permissions_userId_permissionId` (Unique Compound).

9. Collection: `role_permissions`

- **Mục đích nghiệp vụ:** Bảng nối thể hiện quyền hạn gắn liền với từng vai trò.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 54,
  "roleId": 2,
  "permissionId": 8
}
```

- **Chỉ mục:** `uk_role_permissions_roleId_permissionId` (Unique Compound).

3.3. Nhóm 3: Danh Mục Sách & Kho Bản Sao Vật Lý (Catalog & Inventory Collections)

10. Collection: `books`

- **Mục đích nghiệp vụ:** Chứa thông tin gốc của các đầu sách được phát hành.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 10,
```

```

"title": "Thiết Kế Cơ Sở Dữ Liệu Nâng Cao",
"author": "GS. Nguyễn Văn A",
"publisher": "NXB Khoa Học & Kỹ Thuật",
"publicationYear": 2025,
"isbn": "978-604-67-1234-5",
"price": 185000,
"rentalPricePerDay": 5000,
"depositPercent": 70,
"summary": "Giáo trình nghiên cứu chuyên sâu về NoSQL, MongoDB và tối ưu truy vấn dữ liệu lớn.",
"coverImageUrl": "/media/covers/advanced-db.jpg",
"isActive": true,
"shelf": "BESTSELLERS",
"version": 4,
"createdAt": "2026-07-10T09:00:00.000Z",
"updatedAt": "2026-09-02T15:30:00.000Z"
}

```

- **Chỉ mục (Indexes):**
- `uk_books_isbn` (Unique, Sparse): Đảm bảo mã chuẩn quốc tế ISBN là duy nhất.
- `idx_books_isActive_createdAt` : Tối ưu hóa truy vấn danh sách sách đang kinh doanh sắp xếp theo sách mới nhất.

11. Collection: `categories`

- **Mục đích nghiệp vụ:** Thể loại phân loại sách (Công nghệ, Văn học, Kinh tế...).
- **Cấu trúc Document mẫu:**

```

{
  "_id": 3,
  "name": "Công Nghệ Thông Tin",
  "slug": "cong-nghe-thong-tin",
  "description": "Sách chuyên ngành lập trình, mạng và dữ liệu"
}

```

- **Chỉ mục (Indexes):**
- `uk_categories_name` (Unique): Tên thể loại không trùng lặp.
- `uk_categories_slug` (Unique, Sparse): Tối ưu đường dẫn SEO.

12. Collection: `book_categories`

- **Mục đích nghiệp vụ:** Bảng nối thể hiện quan hệ N - N giữa Sách và Thể loại.
- **Cấu trúc Document mẫu:**

```

{
  "_id": 45,
  "bookId": 10,
  "categoryId": 3
}

```

- **Chỉ mục (Indexes):**
- `idx_book_categories_categoryId_bookId` : Phục vụ truy vấn lấy tất cả các sách thuộc một danh mục cụ thể.
- `uk_book_categories_bookId_categoryId` (Unique Compound): Chống gán trùng thể loại cho sách.

13. Collection: `book_copies`

- **Mục đích nghiệp vụ:** Quản lý từng cuốn sách vật lý cụ thể nằm trên kệ kho.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 1001,
  "bookId": 10,
  "status": "AVAILABLE",
  "condition": "NEW",
  "notes": "Bản in lần thứ 3, tem niêm phong nguyên vẹn",
  "_mongoLock": null,
  "createdAt": "2026-07-12T10:00:00.000Z"
}
```

- **Chỉ mục (Indexes):**
- `idx_book_copies_bookId_status` : Cực kỳ quan trọng để quét nhanh các cuốn sách còn trống (AVAILABLE) cho thuê.

3.4. Nhóm 4: Giỏ Hàng & Khuyến Mãi (Cart & Voucher Collections)

14. Collection: `carts`

- **Mục đích nghiệp vụ:** Giỏ hàng độc nhất của từng tài khoản người dùng.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 55,
  "userId": 1,
  "createdAt": "2026-08-10T11:00:00.000Z",
  "updatedAt": "2026-09-04T08:20:00.000Z"
}
```

- **Chỉ mục:** `uk_carts_userId` (Unique, Sparse): Mỗi khách hàng có tối đa 1 giỏ hàng.

15. Collection: `cart_items`

- **Mục đích nghiệp vụ:** Các mặt hàng trong giỏ, hỗ trợ cả hàng mua và hàng thuê có kỳ hạn.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 204,
  "cartId": 55,
  "bookId": 10,
  "itemType": "RENTAL",
  "quantity": 1,
  "rentalTermValue": 2,
  "rentalTermUnit": "MONTH",
  "depositAmount": 129500,
  "rentalPrice": 300000,
  "createdAt": "2026-09-04T08:20:00.000Z"
}
```

- **Chỉ mục:** `uk_cart_items_cartId_bookId_itemType_rentalTermValue_rentalTermUnit` (Unique Compound): Tự động gộp dòng khi thêm sản phẩm trùng loại và kỳ hạn.

16. Collection: `vouchers`

- **Mục đích nghiệp vụ:** Mã khuyến mãi giảm giá đơn hàng.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 5,
  "code": "DAIHOC2026",
  "discountType": "PERCENTAGE",
  "discountValue": 15,
  "maxDiscountAmount": 50000,
  "minOrderValue": 200000,
  "startAt": "2026-09-01T00:00:00.000Z",
  "endAt": "2026-09-30T23:59:59.000Z",
  "maxUsages": 1000,
  "usedCount": 142,
  "maxUsagesPerUser": 1,
  "status": "ACTIVE"
}
```

- **Chỉ mục (Indexes):**
- `uk_vouchers_code` (Unique): Mã voucher không trùng lặp.
- `idx_vouchers_status_startAt_endAt` : Tối ưu hóa quét các voucher đang trong thời gian hữu hiệu.

17. Collection: `voucher_usages`

- **Mục đích nghiệp vụ:** Nhật ký áp dụng voucher theo cơ chế giữ chỗ - chốt lượt dùng (Two-phase commit).
- **Cấu trúc Document mẫu:**

```
{
  "_id": 890,
  "voucherId": 5,
  "userId": 1,
  "orderId": 78,
  "discountAmount": 45000,
  "status": "COMMITTED",
  "usedAt": "2026-09-04T08:35:00.000Z"
}
```

- **Chỉ mục:** `idx_voucher_usages_userId_voucherId` : Kiểm tra số lần người dùng đã áp dụng voucher này.

3.5. Nhóm 5: Đơn Hàng & Lịch Sử Trạng Thái (Order Collections)

18. Collection: `orders`

- **Mục đích nghiệp vụ:** Thực thể Đơn hàng trung tâm.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 78,
  "orderCode": "ORD-20260904-78A1",
  "userId": 1,
  "orderType": "MIXED",
  "status": "CONFIRMED",
  "paymentStatus": "PAID",
  "paymentMethod": "BANK_TRANSFER",
  "subtotalAmount": 485000,
  "discountAmount": 45000,
  "shippingFee": 25000,
  "totalAmount": 465000,
  "shippingAddress": {
    "recipientName": "Nguyễn Văn Đọc",
    "phoneNumber": "0912345678",
    "fullAddress": "Số 298 đường Cầu Diễn, Phường Minh Khai, Quận Bắc Từ Liêm, Hà Nội"
  },
  "note": "Giao hàng trong giờ hành chính",
  "version": 2,
  "createdAt": "2026-09-04T08:30:00.000Z",
  "updatedAt": "2026-09-04T08:35:15.000Z"
}
```

- **Chỉ mục (Indexes):**
- `uk_orders_orderCode` (Unique): Tra cứu đơn hàng theo mã đơn độc nhất.
- `idx_orders_userId_createdAt` : Lấy lịch sử mua hàng của khách hàng theo thứ tự thời gian mới nhất.

19. Collection: `order_items`

- **Mục đích nghiệp vụ:** Chi tiết từng món hàng trong đơn (được tách thành collection riêng để tránh vượt kích thước Document).
- **Cấu trúc Document mẫu:**

```
{
  "_id": 150,
  "orderId": 78,
  "bookId": 10,
  "itemType": "PURCHASE",
  "quantity": 2,
  "unitPrice": 185000,
  "depositAmount": 0,
  "rentalTermValue": null,
  "rentalTermUnit": null,
  "totalPrice": 370000
}
```

- **Chỉ mục:** `idx_order_items_orderId` : Lấy nhanh toàn bộ sản phẩm thuộc đơn hàng.

20. Collection: `order_status_history`

- **Mục đích nghiệp vụ:** Sổ nhật ký kiểm toán bất biến (Audit trail) của đơn hàng.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 312,
  "orderId": 78,
  "status": "CONFIRMED",
  "source": "PAYMENT_TIMO_IMAP",
  "note": "Khớp lệnh tự động từ thông báo biến động số dư ngân hàng",
  "changedAt": "2026-09-04T08:35:15.000Z"
}
```

- **Chỉ mục:** `idx_order_status_history_orderId_changedAt` : Hiển thị dòng thời gian tiến độ giao hàng trên ứng dụng.

3.6. Nhóm 6: Thanh Toán & Đối Soát Ngân Hàng (Payment & Reconciliation Collections)

21. Collection: `payments`

- **Mục đích nghiệp vụ:** Quản lý giao dịch thanh toán cho đơn hàng hoặc gói hội viên.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 77,
  "orderId": 78,
  "subscriptionId": null,
  "amount": 465000,
  "method": "BANK_TRANSFER",
  "status": "SUCCESS",
  "transferReference": "PAY78X9A2",
  "bankTransactionReference": "FT262489812457",
  "gatewayProvider": "TIMO_IMAP",
  "expiresAt": "2026-09-04T09:00:00.000Z",
  "createdAt": "2026-09-04T08:30:00.000Z"
}
```

- **Chỉ mục (Indexes):**
- `uk_payments_transferReference` (Unique, Sparse): Khóa tìm kiếm cốt lõi của IMAP Poller khi giải mã nội dung chuyển khoản.
- `idx_payments_orderId_createdAt` : Tra cứu lịch sử thanh toán của đơn hàng.

22. Collection: `processed_bank_messages`

- **Mục đích nghiệp vụ:** Lưu trữ định danh các email và giao dịch ngân hàng đã xử lý, ngăn chặn việc khớp tiền 2 lần (Idempotency Key).
- **Cấu trúc Document mẫu:**

```
{
  "_id": 401,
  "messageId": "<CAG_xyz123@mail.gmail.com>",
  "bankTxnRef": "FT262489812457",
  "processedAt": "2026-09-04T08:35:14.000Z"
}
```

- **Chỉ mục (Indexes):**
- `uk_processed_bank_messages_messageId` (Unique): Chống trùng lặp email.
- `uk_processed_bank_messages_bankTxnRef` (Unique, Sparse): Chống trùng lặp mã giao dịch ngân hàng.

23. Collection: `unmatched_transfers`

- **Mục đích nghiệp vụ:** Vùng cách ly (Quarantine) chứa các khoản tiền khách chuyển khoản nhưng không khớp tự động được với bất kỳ đơn hàng nào.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 12,
  "messageId": "<CAG_abc456@mail.gmail.com>",
  "bankTxnRef": "FT262489998811",
  "paymentReference": "PAY99ZZZ",
  "amount": 500000,
  "receivedAt": "2026-09-04T09:15:00.000Z",
  "reason": "No matching pending payment with exact amount",
  "createdAt": "2026-09-04T09:16:00.000Z"
}
```

- **Chỉ mục:** `idx_unmatched_transfers_createdAt` : Hiển thị các khoản tiền treo mới nhất cho kế toán viên xử lý.

3.7. Nhóm 7: Thuê Sách & Gói Hội Viên Đọc Sách (Rental & Subscription Collections)

24. Collection: `rentals`

- **Mục đích nghiệp vụ:** Quản lý từng hợp đồng thuê sách vật lý đang diễn ra.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 66,
  "orderId": 151,
  "bookCopyId": 1001,
  "userId": 1,
  "termUnit": "MONTH",
  "termValue": 2,
  "depositAmount": 129500,
  "startDate": "2026-09-04",
  "plannedReturnDate": "2026-11-04",
  "actualReturnDate": null,
  "status": "RENTING",
  "refundedDeposit": 0,
  "lateFee": 0,
  "damageFee": 0,
  "createdAt": "2026-09-04T08:35:20.000Z"
}
```

- **Chỉ mục (Indexes):**
- `uk_rentals_orderItemId` (Unique): Mỗi mục thuê trong đơn chỉ kích hoạt duy nhất một hợp đồng thuê.
- `idx_rentals_userId_createdAt` : Lấy danh sách sách đang thuê của độc giả.

- `idx_rentals_actualReturnDate_plannedReturnDate_status` : Cực kỳ quan trọng để quét danh sách các sách **quá hạn trả** (`GetOverdueRentalsService`).

25. Collection: `rental_fulfillments`

- **Mục đích nghiệp vụ:** Hàng đợi đảm bảo giao việc gán sách vật lý cho đơn thuê luôn bền vững ngay cả khi hệ thống khởi động lại.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 80,
  "orderId": 78,
  "status": "COMPLETED",
  "retryCount": 0,
  "lastError": null,
  "updatedAt": "2026-09-04T08:35:20.000Z"
}
```

- **Chỉ mục (Indexes):**
- `uk_rental_fulfillments_orderId` (Unique): Mỗi đơn hàng có tối đa một tiến trình hoàn tất thuê.
- `idx_rental_fulfillments_status_updatedAt` : Quét các tác vụ thuê bị lỗi để thực hiện cơ chế Retry tự động.

26. Collection: `subscriptions`

- **Mục đích nghiệp vụ:** Định nghĩa các gói hội viên đọc sách (Standard, VIP, Premium).
- **Cấu trúc Document mẫu:**

```
{
  "_id": 1,
  "name": "Gói Hội Viên Đọc Sách VIP 6 Tháng",
  "durationDays": 180,
  "price": 599000,
  "maxConcurrentRentals": 5,
  "isDepositFree": true,
  "isActive": true
}
```

27. Collection: `customer_subscriptions`

- **Mục đích nghiệp vụ:** Hợp đồng gói hội viên thực tế của khách hàng.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 23,
  "userId": 1,
  "subscriptionId": 1,
  "startDate": "2026-09-01",
  "endDate": "2027-03-01",
  "status": "ACTIVE",
  "paymentId": 65,
  "createdAt": "2026-09-01T10:00:00.000Z"
}
```

3.8. Nhóm 8: Tương Tác, Hỗ Trợ, Đánh Giá & Truyền Thông (Engagement & Content Collections)

28. Collection: `book_reviews`

- **Mục đích nghiệp vụ:** Đánh giá và nhận xét về đầu sách từ khách hàng đã trải nghiệm thực tế.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 88,
  "bookId": 10,
  "userId": 1,
  "orderId": 150,
  "rating": 5,
  "title": "Cuốn sách tuyệt vời về cơ sở dữ liệu nâng cao",
  "content": "Giải thích rất cặn kẽ về WiredTiger và cách MongoDB thực thi giao dịch phân tán.",
  "reviewSource": "PURCHASE",
  "createdAt": "2026-09-10T16:00:00.000Z"
}
```

- **Chỉ mục:** `uk_book_reviews_userId_orderItemId` (Unique Compound): Khách hàng chỉ được đánh giá 1 lần cho mỗi món hàng đã mua.

29. Collection: `support_conversations`

- **Mục đích nghiệp vụ:** Kênh hội thoại hỗ trợ khách hàng giữa Độc giả và Nhân viên nhà sách.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 34,
  "userId": 1,
  "staffUnreadCount": 0,
  "customerUnreadCount": 1,
  "lastMessagePreview": "Dạ em đã kiểm tra và xác nhận sách của anh rồi ạ.",
  "assignedStaffUserId": 2,
  "lastMessageAt": "2026-09-04T10:30:00.000Z",
  "createdAt": "2026-08-15T09:00:00.000Z"
}
```

- **Chỉ mục (Indexes):**
- `uk_support_conversations_userId` (Unique): Mỗi khách hàng có duy nhất một luồng hội thoại hỗ trợ.
- `idx_support_conversations_lastMessageAt` : Sắp xếp các cuộc trò chuyện có tin nhắn mới nhất lên đầu danh sách của nhân viên.

30. Collection: `support_messages`

- **Mục đích nghiệp vụ:** Từng tin nhắn trao đổi trong luồng hội thoại.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 501,
  "conversationId": 34,
  "sender": "STAFF",
```

```

"senderUserId": 2,
"body": "Dạ em đã kiểm tra và xác nhận sách của anh rồi ạ.",
"createdAt": "2026-09-04T10:30:00.000Z"
}

```

- **Chỉ mục:** `idx_support_messages_conversationId_createdAt` : Tài tin nhắn theo thứ tự thời gian tăng dần trong phòng chat.

31. Collection: `support_message_attachments`

- **Mục đích nghiệp vụ:** Tập hình ảnh đính kèm theo tin nhắn hỗ trợ.
- **Cấu trúc Document mẫu:**

```

{
  "_id": 88,
  "messageId": 501,
  "imageUrl": "/media/support/bien-lai-chuyen-tien-78.jpg",
  "originalName": "bien-lai.jpg",
  "contentType": "image/jpeg",
  "createdAt": "2026-09-04T10:30:00.000Z"
}

```

- **Chỉ mục:** `idx_support_message_attachments_messageId__id` : Truy xuất toàn bộ ảnh đính kèm của một danh sách tin nhắn.

32. Collection: `user_notifications`

- **Mục đích nghiệp vụ:** Hộp thư thông báo in-app gửi tới khách hàng.
- **Cấu trúc Document mẫu:**

```

{
  "_id": 770,
  "userId": 1,
  "type": "PAYMENT",
  "title": "Thanh toán thành công",
  "body": "Đơn hàng ORD-20260904-78A1 đã được xác nhận thanh toán qua Timo IMAP.",
  "targetPath": "/don-hang/78",
  "readAt": null,
  "createdAt": "2026-09-04T08:35:16.000Z"
}

```

- **Chỉ mục (Indexes):**
- `idx_user_notifications_userId_createdAt` : Lấy danh sách thông báo mới nhất.
- `idx_user_notifications_userId_readAt` : Tối ưu hóa đếm nhanh số lượng thông báo chưa đọc (`unreadCount`).

33. Collection: `push_subscriptions`

- **Mục đích nghiệp vụ:** Lưu trữ thông tin đăng ký nhận thông báo đẩy Web Push (VAPID) của thiết bị trình duyệt.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 44,
  "userId": 1,
  "endpoint": "https://fcm.googleapis.com/fcm/send/e89Xz...",
  "p256dh": "BNcR...",
  "auth": "tBH...",
  "createdAt": "2026-08-20T11:00:00.000Z"
}
```

- **Chỉ mục:** `uk_push_subscriptions_userId_endpoint` (Unique Compound): Tránh lưu trùng lặp endpoint của cùng một thiết bị.

34. Collection: `blog_posts`

- **Mục đích nghiệp vụ:** Các bài viết tin tức, bài đánh giá và giới thiệu sách biên tập.
- **Cấu trúc Document mẫu:**

```
{
  "_id": 15,
  "title": "Top 5 Cuốn Sách Cơ Sở Dữ Liệu Nâng Cao Dành Cho Học Viên Cao Học",
  "slug": "top-5-cuon-sach-co-so-du-lieu-nang-cao",
  "summary": "Tổng hợp các tác phẩm kinh điển về cơ sở dữ liệu phân tán, NoSQL và MongoDB.",
  "content": "# Giới thiệu\nTrong thời đại dữ liệu lớn...",
  "coverImageUrl": "/media/blog/database-books.jpg",
  "authorUserId": 4,
  "status": "PUBLISHED",
  "publishedAt": "2026-09-01T08:00:00.000Z",
  "createdAt": "2026-08-28T14:00:00.000Z"
}
```

- **Chỉ mục (Indexes):**
- `uk_blog_posts_slug` (Unique): Đường dẫn URL bài viết không bao giờ được trùng.
- `idx_blog_posts_status_publishedAt` : Tối ưu hóa truy vấn bài viết công khai trên trang chủ.

4. SO SÁNH CÁC MẪU TRUY VẤN ĐIỂN HÌNH: SQL QUERY VS MONGODB AGGREGATION

Để giúp kỹ sư SQL nắm bắt cách MongoDB vận hành trong dự án, dưới đây là sự đối chiếu giữa câu lệnh SQL truyền thống và cách hiện thực hóa trong Velstrong Bookstore:

4.1. Bài toán: Tìm Top 10 cuốn sách bán chạy nhất trong 30 ngày gần đây

- **Câu lệnh trong SQL (Postgres / MySQL):**

```
SELECT oi.book_id, SUM(oi.quantity) AS total_sales
FROM order_items oi
JOIN orders o ON oi.order_id = o.id
WHERE o.status IN ('CONFIRMED', 'PROCESSING', 'SHIPPING', 'COMPLETED')
  AND o.created_at >= NOW() - INTERVAL '30 days'
  AND oi.item_type = 'PURCHASE'
GROUP BY oi.book_id
ORDER BY total_sales DESC
LIMIT 10;
```

- Cách thực hiện trong MongoDB của dự án (`MongoOrderItemPersistenceAdapter`):

```
// Bước 1: Quét các đơn hàng hợp lệ trong thời gian quy định
List<Long> validOrderIds = mongo.find(Query.query(new Criteria().andOperator(
    Criteria.where("status").in(List.of("CONFIRMED", "PROCESSING", "SHIPPING", "COMPLETED")),
    Criteria.where("createdAt").gte(since))), Document.class, "orders").stream()
    .map(value -> ((Number) value.get("_id")).longValue()).toList();

// Bước 2: Quét các mục đơn hàng tương ứng và gom nhóm trong bộ nhớ (Memory grouping)
Map<Long, Long> quantities = new LinkedHashMap<>();
find("order_items", OrderItemJpaEntity.class,
Query.query(Criteria.where("orderId").in(validOrderIds)))
    .forEach(value -> quantities.merge(value.getBookId(), value.getQuantity().longValue(),
Long::sum));

// Bước 3: Sắp xếp giảm dần và lấy top limit
return quantities.entrySet().stream()
    .sorted(Map.Entry.<Long, Long>comparingByValue(Comparator.reverseOrder()))
    .limit(limit)
    .map(entry -> new BookSalesCount(entry.getKey(), entry.getValue())).toList();
```

- Tương đương bằng Aggregation Pipeline nếu viết thuần trong MongoDB:

```
db.orders.aggregate([
  { $match: {
    status: { $in: ["CONFIRMED", "PROCESSING", "SHIPPING", "COMPLETED"] },
    createdAt: { $gte: ISODate("2026-08-04T00:00:00Z") }
  }},
  { $lookup: {
    from: "order_items",
    localField: "_id",
    foreignField: "orderId",
    as: "items"
  }},
  { $unwind: "$items" },
  { $match: { "items.itemType": "PURCHASE" } },
  { $group: {
    _id: "$items.bookId",
    totalSales: { $sum: "$items.quantity" }
  }},
  { $sort: { totalSales: -1 } },
  { $limit: 10 }
]);
```

4.2. Bài toán: Thống kê số lượng đơn hàng theo trạng thái của một khách hàng

- SQL Query:

```
SELECT status, COUNT(*)
FROM orders
WHERE user_id = 1
GROUP BY status;
```

- MongoDB Query trong `MongoOrderPersistenceAdapter` :

```
Map<OrderStatus, Long> counts = new EnumMap<>(OrderStatus.class);
mongo.find(Query.query(Criteria.where("userId").is(userId)), Document.class, "orders")
    .forEach(value -> {
        String status = value.getString("status");
        if (status != null) counts.merge(OrderStatus.valueOf(status), 1L, Long::sum);
    });
```

5. CẨM NANG THỰC CHIẾN & QUY TẮC THIẾT KẾ CHO KỸ SƯ SQL

Khi thiết kế và bảo trì hệ thống MongoDB, kỹ sư xuất thân từ thế giới SQL cần ghi nhớ các nguyên tắc vàng sau:

1. **Quy tắc ESR (Equality, Sort, Range) trong thiết kế Compound Index:** - Khi tạo một chỉ mục phức hợp gồm nhiều trường, thứ tự khai báo các trường quyết định hiệu năng:

1. **E - Equality:** Các trường dùng để so sánh bằng (`is(...)` hoặc `$eq`) phải đứng ĐẦU TIÊN.
2. **S - Sort:** Các trường dùng để sắp xếp (`orderBy(...)` hoặc `$sort`) phải đứng TIẾP THEO.
3. **R - Range:** Các trường dùng để lọc theo khoảng (`gte` , `lte` , `in` , `$regex`) phải đứng CUỐI CÙNG. - Ví dụ: Truy vấn tìm sách: `where isActive = true order by createdAt desc` . Index chuẩn phải là: `idx_books_isActive_createdAt` (`isActive` đứng trước `createdAt`). Nếu đảo ngược thứ tự, MongoDB sẽ không thể vừa lọc vừa sắp xếp trên cùng một lượt quét B-Tree.

2. **Cảnh giác với Anti-pattern: "Mảng Vô Hạn" (Unbounded Arrays):** - Trong SQL, bạn thoải mái thêm hàng triệu dòng con liên kết khóa ngoại với bảng cha. - Trong MongoDB, nếu bạn nhúng mảng `items` hoặc `logs` vào trong 1 document cha, document đó sẽ tăng dần kích thước. Nếu vượt quá **16 MB**, MongoDB sẽ ném lỗi `DocumentTooLarge` và từ chối ghi. Hơn nữa, việc document liên tục phình to sẽ khiến WiredTiger phải di dời document trên đĩa, gây phân mảnh dữ liệu trầm trọng. - **Giải pháp:** Với các quan hệ có khả năng tăng trưởng liên tục theo thời gian (như `order_items` , `order_status_history` , `support_messages`), hãy tách thành collection riêng và tham chiếu bằng `orderId` hoặc `conversationId` như dự án Velstrong Bookstore đã làm.

3. **Luôn sử dụng Replica Set ngay cả trong môi trường phát triển (Local Development):** - Nhiều lập trình viên thường khởi động MongoDB độc lập (Standalone) trên máy tính cá nhân và bần khoăn vì sao Spring ném lỗi `Cannot start transaction: Standalone servers do not support multi-document transactions` . - Trong `docker-compose.yml` của dự án, MongoDB luôn được cấu hình chạy dưới dạng Replica Set:

```
command: ["mongod", "--replSet", "rs0", "--bind_ip_all"]
```

- Điều này đảm bảo môi trường Dev hoàn toàn giống với Production, sẵn sàng cho các giao dịch `@Transactional` trên nhiều collection.
4. **Tránh bẫy N+1 Query khi thực hiện Application-level Joins:** - Khi tách thành các collection riêng biệt và truy vấn bằng mã ứng dụng, tuyệt đối không được viết vòng lặp `for` để gọi `findById()` cho từng phần tử. - Hãy luôn gom nhóm các ID lại thành danh sách và dùng toán tử `where("_id").in(ids)` (tương đương `WHERE id IN (...)` trong SQL) để chỉ tốn duy nhất 1 lần round-trip tới cơ sở dữ liệu.